```
#include <iostream>
#include <string>
#include <cstdint>
#include <cstring>
#include <vulkan/vulkan.h>

#define OPCODE_RV_SVE 0b1011011
#define FUNCT3_VMUL_VV 0b010
#define FUNCT7_VECTOR 0b0000000

#define RV_SVE_ENCODE(f7, rs2, rs1, f3, rd, opc) \
(((uint64_t)(f7) << 25) | ((uint64_t)(rs2) << 20) | ((uint64_t)(rs1) << 15) | \
((uint64_t)(f3) << 12) | ((uint64_t)(rd) << 7) | ((uint64_t)(opc)))

using namespace std;

// ===== 64bit RV-SVE 명령어 구조 =====
struct Uop64 {
    uint64_t raw_inst;
    uint64_t rd, rs1, rs2, funct3;
    bool is_vector;
    string opcode;

    void decode(uint64_t inst) {
        raw_inst = inst;
        rd = (inst >> 7) & 0x1F;
        rs1 = (inst >> 15) & 0x1F;
        rs2 = (inst >> 20) & 0x1F;
        funct3 = (inst >> 12) & 0x7;
        is_vector = ((inst & 0x7F) == OPCODE_RV_SVE);
        opcode = (is_vector && funct3 == FUNCT3_VMUL_VV) ? "VMUL.VV" : "NOP";
    }
};

// ===== 64bit AP 컨텍스트 =====
struct AP64Context {
    Uop64 uop;
    bool actionRequired;
    uint64_t predictedLoad;
    string priority;
    string renderToken;
};

// ===== Vulkan 기반 64bit 드라이버 =====
class Gemini64Driver {
public:
    Gemini64Driver(VkDevice device, VkCommandBuffer cmd, VkPipelineLayout layout)
        : m_device(device), m_cmd(cmd), m_layout(layout) {}
```

```
    void pushConstants(const float* matrix64) {
        vkCmdPushConstants(m_cmd, m_layout, VK_SHADER_STAGE_VERTEX_BIT, 0,
sizeof(float) * 16, matrix64);
        cout << "[Gemini64] PushConstants → GPU\n";
    }

    void buildComputePipeline(VkShaderModule shaderModule) {
        VkPipelineShaderStageCreateInfo stage{};
        stage.sType =
VK_STRUCTURE_TYPE_PIPELINE_SHADER_STAGE_CREATE_INFO;
        stage.stage = VK_SHADER_STAGE_COMPUTE_BIT;
        stage.module = shaderModule;
        stage.pName = "main";

        VkComputePipelineCreateInfo info{};
        info.sType = VK_STRUCTURE_TYPE_COMPUTE_PIPELINE_CREATE_INFO;
        info.stage = stage;
        info.layout = m_layout;

        vkCreateComputePipelines(m_device, VK_NULL_HANDLE, 1, &info, nullptr,
&m_pipeline);
        cout << "[Gemini64] ComputePipeline Built\n";
    }

    void dispatch(uint64_t vertexCount64) {
        uint64_t groupCount = (vertexCount64 + 255) / 256;
        vkCmdBindPipeline(m_cmd, VK_PIPELINE_BIND_POINT_COMPUTE, m_pipeline);
        vkCmdDispatch(m_cmd, static_cast<uint32_t>(groupCount), 1, 1);
        cout << "[Gemini64] Dispatch " << groupCount << " groups\n";
    }

    void submit(VkQueue queue) {
        VkSubmitInfo submit{};
        submit.sType = VK_STRUCTURE_TYPE_SUBMIT_INFO;
        submit.commandBufferCount = 1;
        submit.pCommandBuffers = &m_cmd;
        vkQueueSubmit(queue, 1, &submit, VK_NULL_HANDLE);
        cout << "[Gemini64] Submitted to GPU queue\n";
    }

private:
    VkDevice m_device;
    VkCommandBuffer m_cmd;
    VkPipelineLayout m_layout;
    VkPipeline m_pipeline;
};
```

```
// ===== GPGPU-style 64bit 파이프라인 =====
class SmartRender64Pipeline {
public:
    AP64Context evaluate(AP64Context ctx) {
        ctx.actionRequired = true;
        return ctx;
    }

    AP64Context render(AP64Context ctx) {
        if (ctx.actionRequired) ctx.renderToken = "RENDER64_TOKEN";
        return ctx;
    }

    AP64Context simulate(AP64Context ctx) {
        ctx.predictedLoad = (ctx.uop.rs1 + ctx.uop.rs2) * 64;
        return ctx;
    }

    AP64Context rank(AP64Context ctx) {
        ctx.priority = (ctx.predictedLoad > 4096) ? "high" : "low";
        return ctx;
    }

    AP64Context dispatch(AP64Context ctx) {
        cout << "[SmartRender64] Dispatch: " << ctx.renderToken << " → " << ctx.priority <<
endl;
        return ctx;
    }

    AP64Context trace(AP64Context ctx) {
        cout << "[SmartRender64] Trace: " << ctx.uop.opcode << " r" << ctx.uop.rd
             << " <- r" << ctx.uop.rs1 << " * r" << ctx.uop.rs2 << endl;
        return ctx;
    }

    AP64Context defer(AP64Context ctx) {
        if (ctx.priority != "high")
            cout << "[SmartRender64] Deferred to idle thread\n";
        return ctx;
    }

    AP64Context run(AP64Context ctx) {
        return defer(trace(dispatch(rank(simulate(render(evaluate(ctx)))))));
    }
};

// ===== 테스트 =====
int main() {
```

```
    VkDevice device = VK_NULL_HANDLE;
    VkCommandBuffer cmd = VK_NULL_HANDLE;
    VkPipelineLayout layout = VK_NULL_HANDLE;
    VkQueue queue = VK_NULL_HANDLE;

    Gemini64Driver driver(device, cmd, layout);
    SmartRender64Pipeline pipeline;

    // RV-SVE 64bit 명령어 생성
    uint64_t inst64 = RV_SVE_ENCODE(FUNCT7_VECTOR, 2, 1, FUNCT3_VMUL_VV, 0,
OPCODE_RV_SVE);
    AP64Context ctx;
    ctx.uop.decode(inst64);

    // 파이프라인 실행
    ctx = pipeline.run(ctx);

    // GPU 드라이버 실행
    float dummyMatrix64[16] = {0};
    driver.pushConstants(dummyMatrix64);
    driver.buildComputePipeline(VK_NULL_HANDLE); // 셰이더 모듈 가정
    driver.dispatch(1024);
    driver.submit(queue);

    return 0;
}
```